Cross-Scale Attention for Long-Term Time Series Forecasting

Liangjian Wen, Quan Hu, Cong Guo, Ao Hu, and Mingyi Zhang

***Abstract*—Transformer-based models, especially PatchTST, have demonstrated remarkable success in time series forecasting tasks. However, the unique nature of time series data, which often contains jitter, and noise, and has inherently lower information density compared to images and texts, poses significant challenges. Specifically, the ViT-inspired patching design is suboptimal for time series data due to the sparse semantic relationships in such data. Moreover, modelling these sparse semantic relationships requires more resources and longer processing times. To overcome these limitations, we introduce a novel approach that leverages cross-scale attention interaction via a multi-scale patching technique. Our method initially treats the entire sequence as a single patch, then progressively divides it into increasing patches, doubling each time. This strategy improves the information density within each patch and reduces the total number of patches needed to model the time series to typically just seven effectively. We design a single layer of attention to model the cross-scale relationships among patches. These qualities significantly enhance computational efficiency. Extensive experiments have shown that our method surpasses or closely approaches existing methods in time-series forecasting benchmarks. Additionally, it achieves a speed that is 12x times faster than PatchTST on the large dataset.**

***Index Terms*—Long-term time series forecasting (LTSF), transformer-based methods, cross-scale interaction.**

I. Introduction

TIME series forecasting plays a pivotal role in multiple domains, encompassing transportation [1], [2], meteorology, finance, energy, healthcare economics, and environmental science [3], [4]. Recently, Transformers [5] are now being extensively harnessed for time series forecasting [6], [7], [8], [9], inspired by their unparalleled success in a variety of domains, including sequence modeling in Computer Vision [10] and Natural Language Processing [11].

Manuscript received 15 June 2024; revised 28 July 2024; accepted 29 July 2024. Date of publication 6 August 2024; date of current version 7 October 2024. This work was supported in part by the Sichuan Science and Technology Department Key Research and Development Project under Grant 2024YFHZ0062, in part by the Special Science Foundation of Sichuan Province for Artificial Intelligence under Grant 2024ZDZX0002[16] and Grant 2024NSFTD0054, and in part by the Fundamental Research Funds for the Central Universities under Grant JBK2103012. The associate editor coordinating the review of this article and approving it for publication was Mrs. Audrey Giremus. *(Corresponding author: Mingyi Zhang.)*

Liangjian Wen, Cong Guo, and Ao Hu are with the School of Computing and Artificial Intelligence and the Research Institute for Digital Economy and Interdisciplinary Sciences, Southwestern University of Finance and Economics, Chengdu 611130, China.

Quan Hu is with the School of Statistics, Southwestern University of Finance and Economics, Chengdu 611130, China.

Mingyi Zhang is with the Department of Biotherapy, West China Hospital, Sichuan University, Chengdu 610065, China (e-mail: zhangmingyi319@scu.edu.cn).

Digital Object Identifier 10.1109/LSP.2024.3439103

Despite their success, recent studies question the Transformer-based models in the realm of time series forecasting. A. Zeng [12] finds simpler, channel-independent linear models can surpass the complexities of Transformer architectures [6], [8], [13] in both performance and efficiency metrics. It is argued that the Transformer-based model is notably effective in extracting semantic relationships in long sequences. However, this contrasts with the primarily numerical and less semantically rich relationships that are observed among time points. Drawing inspiration from Vision Transformers (ViT) [10], the PatchTST [9] models segment time series data into small patches at the subseries level, which are then utilized as input tokens for a Transformer-based model. Following the channel-independent setting in [12], [14], it has achieved better performance. The potential of Transformer-based models remains significant in the domain of time series forecasting.

However, unlike images and texts, time series data have lower information density. As a result, these small patches of PatchTST contain limited effective information, leading to fewer semantic relationships among patches. Moreover, time series data are frequently afflicted by substantial noise and jitter, which significantly distort and obscure the underlying patterns and trends. These inherent characteristics of time series data present challenges in capturing correlations among different patches through cross-time interaction. To capture temporal correlations effectively, PatchTST employs a larger Transformer-base model designed to better discern intricate patterns and relationships. However, this results in a significant computational burden, requiring more resources and longer processing times.

In light of these challenges, we propose a cross-scale attention method instead of cross-time interaction, which tackles time series forecasting problems from a brand new perspective of cross-scale interaction. The key component of our method is a multi-scale patching technique designed for the cross-scale of time series data. As shown in Fig. 1, it considers the sequence in its entirety as a singular patch. Subsequently, it divides the sequence into a pair of patches. This division process is further extended, splitting the sequence into four patches, and continues in this manner, progressively increasing the number of patches. Unlike previous multi-scale methods [15], [16], our approach involves large-scale patching, which enhances the patches to contain more semantic information to make it easy to learn the underlying patterns and trends. We also design a residual dense layer to extract useful information from patch embedding for the cross-scale interaction. Finally, we adopt the attention mechanism to model the cross-scale relationships among patches. Compared with PatchTST's cross-time patch interactions, cross-scale interactions among multi-grained patches have a superior

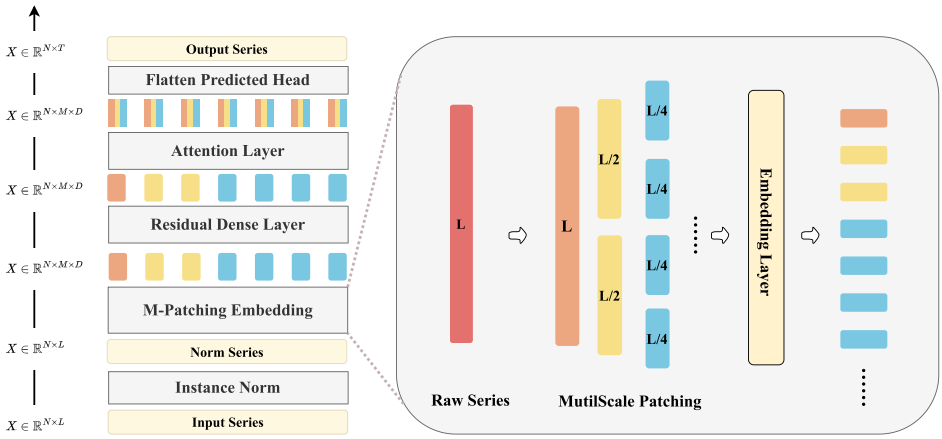

Fig. 1. The architecture of CSformer, which consists of the M-patching embedding, a residual dense layer, an attention layer, and a Flatten Head. Muti-Scale Patching Embedding initially treats the entire sequence as a single patch, then progressively divides it into increasing patches, doubling each time.

capacity for assimilating and comprehending local-global information. Moreover, only seven patches and one attention layer are required in practice, greatly improving computational efficiency.

Our contributions are as follows: (1) We propose a multi-scale patching approach to tackle time series forecasting problems from a brand-new perspective of cross-scale attention. It significantly enhances computational efficiency. (2) We develop the CSformer model, including the Residual Dense Layer, and Attention Layer. The multi-head setting and the feed-forward layer of the Transformer have almost no impact on the predictive performance. (3) The CSformer model surpasses or closely approaches existing methods in time-series forecasting benchmarks. Additionally, it achieves a speed that is 12x times faster than PatchTST on the large dataset.

II. Proposed Method

A. The General Structure

In the context of multivariate time series forecasting, we consider historical data $\mathbf{X} = \{\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_L\} \in \mathbb{R}^{L\times N}$, encompassing L time steps and N variates. Our objective is to predict the subsequent S time steps, represented as $\mathbf{Y} = \{\mathbf{x}_{L+1}, \mathbf{x}_{L+2}, \ldots, \mathbf{x}_{L+S}\} \in \mathbb{R}^{S\times N}$. For the purpose of notational simplicity, let $\mathbf{X}_i$ and $\mathbf{Y}_i$ represent the complete time series corresponding to each variable indexed by i

The overarching framework of our approach is illustrated in Fig. 1. This method utilizes a solely encoder-based architecture, which includes modules such as M-Patching Embedding, the Residual Dense Layer, an Attention Layer, and a Flatten Predicted Head. The process for forecasting future sequences of each distinct variate $\mathbf{Y}_i$, contingent upon the historical series $\mathbf{X}_i$, can be succinctly delineated as follows:

$$\mathbf{PE}_i = \text{M-PatchEmbedding}(\mathbf{X}_i), \tag{1}$$

$$\mathbf{H}_i^R = \text{ResidualDenseLayer}(\mathbf{PE}_i), \tag{2}$$

$$\mathbf{H}_i^A = \text{AttentionLayer}(\mathbf{H}_i^R), \tag{3}$$

$$\hat{\mathbf{Y}}_i = \text{FlattenPredictedHead}(\mathbf{H}_i^A), \tag{4}$$

where $\mathbf{PE}_i = \{\mathbf{z}_1, \mathbf{z}_2, \ldots, \mathbf{z}_m\} \in \mathbb{R}^{M\times D}$ contains M embedded tokens of dimension D.

B. Muti-Scale Patching Embedding

Instead of cross-time interaction, the approach of multi-scale patching addresses the challenges of time series forecasting through the innovative lens of cross-scale interaction. The whole process is depicted in Fig. 1. This technique considers the sequence in its entirety as a singular patch. Subsequently, it divides the sequence into a pair of patches. This division of the process is further extended, splitting the sequence into four patches, and continues in this manner, progressively increasing the number of patches. The iterative division process persists until the frequency of divisions meets a predetermined threshold, designated by the parameter $scale$. Upon reaching this specified criterion, the divisional process is halted.

The cumulative count M of patches is determined to $scale$:

$$M = 1 + 2^1 + \cdots + 2^{\text{scale-1}}, \tag{5}$$

which includes incorporating the entire sequence as an individual patch in this total. In the majority of practical instances, it is observed that merely seven patches are necessitated, significantly diminishing their quantity and enhancing computational efficiency. Furthermore, this approach of large-scale patching not only augments the patches with increased semantic information, facilitating the suppression of jitter and noise but also renders them more apt for integration within Transformer architectures. Following the implementation of multi-scale patching, we define $scale$ linear layers as the embedding layer. Subsequently, as shown in Fig. 1, all patches undergo a transformation process, resulting in embedding vectors of uniform dimensionality.

C. Model Architecture

Redidual Dense Layer: To achieve the cross-scale interaction better, we design a residual dense layer to extract useful information from patch embeddings. Constituting a multi-layer perceptron (MLP) with a singular hidden layer, the residual dense layer employs a Gaussian Error Linear Unit (GELU) activation

TABLE I
FULL RESULTS OF THE LONG-TERM FORECASTING TASK. THE HIGHEST-PERFORMING RESULTS ARE MARKED IN BOLD, AND THE SECOND-HIGHEST ARE UNDERLINED

Models		**CSformer**		FITS		iTransformer		PatchTST		Crossformer		DLinear		FEDformer		Autoformer	
Metric		MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE
Weather	96	**0.149**	**0.197**	0.175	0.227	0.174	0.214	0.152	0.199	0.158	0.230	0.176	0.237	0.238	0.314	0.249	0.329
	192	**0.196**	**0.244**	0.218	0.262	0.221	0.254	0.197	0.243	0.206	0.277	0.220	0.282	0.275	0.329	0.325	0.370
	336	**0.246**	**0.283**	0.265	0.296	0.278	0.296	0.249	0.283	0.272	0.335	0.265	0.319	0.339	0.377	0.351	0.391
	720	0.323	0.335	0.333	0.343	0.358	0.347	**0.320**	0.335	0.398	0.418	0.323	0.362	0.389	0.409	0.415	0.426
Traffic	96	**0.362**	0.254	0.410	0.280	0.395	0.268	0.367	**0.251**	0.522	0.290	0.410	0.282	0.576	0.359	0.597	0.371
	192	**0.384**	0.263	0.422	0.284	0.417	0.276	0.385	**0.259**	0.530	0.293	0.423	0.287	0.610	0.380	0.607	0.382
	336	**0.396**	0.267	0.434	0.290	0.433	0.283	0.398	**0.265**	0.558	0.305	0.436	0.296	0.608	0.375	0.623	0.387
	720	**0.430**	**0.287**	0.464	0.308	0.467	0.302	0.434	**0.287**	0.589	0.328	0.466	0.315	0.621	0.375	0.639	0.395
Electricity	96	**0.130**	**0.222**	0.140	0.236	0.148	0.240	**0.130**	**0.222**	0.219	0.314	0.140	0.237	0.186	0.302	0.196	0.313
	192	**0.148**	**0.240**	0.154	0.248	0.162	0.253	**0.148**	**0.240**	0.231	0.322	0.153	0.249	0.197	0.311	0.211	0.324
	336	**0.165**	**0.257**	0.171	0.264	0.178	0.269	0.167	0.261	0.246	0.337	0.169	0.267	0.213	0.328	0.214	0.327
	720	0.203	**0.290**	0.209	0.296	0.225	0.317	**0.202**	0.291	0.280	0.363	0.203	0.301	0.233	0.344	0.236	0.342
ETTh1	96	0.381	0.403	**0.373**	**0.395**	0.386	0.405	0.379	0.405	0.423	0.448	0.375	0.399	0.376	0.415	0.435	0.446
	192	0.413	0.421	**0.405**	**0.413**	0.441	0.436	0.414	0.421	0.471	0.474	0.412	0.420	0.423	0.446	0.456	0.457
	336	0.434	0.438	**0.428**	**0.427**	0.487	0.458	0.441	0.441	0.570	0.546	0.439	0.443	0.444	0.462	0.486	0.487
	720	0.448	0.470	**0.426**	**0.446**	0.503	0.491	0.457	0.471	0.653	0.621	0.501	0.490	0.469	0.492	0.515	0.517
ETTh2	96	0.277	0.340	**0.274**	**0.337**	0.297	0.349	0.297	0.349	0.745	0.584	0.289	0.353	0.332	0.374	0.332	0.368
	192	0.337	0.379	**0.334**	**0.375**	0.380	0.400	0.338	0.378	0.877	0.656	0.383	0.418	0.407	0.446	0.426	0.434
	336	0.359	0.397	**0.357**	**0.396**	0.428	0.432	0.368	0.340	1.043	0.731	0.448	0.465	0.400	0.447	0.477	0.479
	720	0.392	0.426	**0.384**	**0.422**	0.427	0.445	0.392	0.430	1.104	0.763	0.605	0.551	0.412	0.469	0.453	0.490
ETTm1	96	0.291	0.343	0.303	0.345	0.334	0.368	**0.290**	**0.342**	0.404	0.426	0.299	0.343	0.326	0.390	0.510	0.492
	192	**0.329**	0.366	0.337	0.365	0.377	0.391	0.332	**0.369**	0.450	0.451	0.335	0.365	0.365	0.415	0.514	0.495
	336	**0.364**	0.388	0.371	**0.385**	0.426	0.420	0.366	0.392	0.532	0.515	0.369	0.386	0.392	0.425	0.510	0.492
	720	**0.419**	0.421	0.427	**0.416**	0.491	0.459	0.420	0.424	0.666	0.589	0.425	0.421	0.446	0.458	0.527	0.493
ETTm2	96	0.166	0.257	**0.165**	**0.255**	0.180	0.264	**0.165**	**0.255**	0.287	0.366	0.167	0.260	0.180	0.271	0.205	0.293
	192	0.223	0.295	**0.220**	**0.291**	0.250	0.309	**0.220**	0.292	0.414	0.492	0.224	0.303	0.252	0.318	0.278	0.336
	336	0.276	0.331	**0.273**	**0.326**	0.311	0.348	0.278	0.329	0.597	0.542	0.281	0.342	0.324	0.364	0.343	0.379
	720	0.370	0.387	**0.366**	**0.382**	0.412	0.407	0.367	0.385	1.730	1.042	0.397	0.421	0.410	0.420	0.414	0.419

function to facilitate non-linear transformations. Furthermore, it incorporates a skip connection. To mitigate overfitting and promote model generalizability, dropout is applied to the linear mapping between the hidden layer and the output. Additionally, to improve the network's convergence properties, layer normalization is implemented at the output stage.

Attention Layer: We conduct a step-by-step empirical experimental study on the designed redundancy of the Transformer-based model for cross-scale interaction. Our findings reveal that both the multi-head configuration and the feed-forward layer contribute minimally, if at all, to the predictive efficacy of the model. Additionally, it is observed that increasing the number of attention layers within the model results in only a negligible performance improvement.

We only adopt the attention without the multi-head configuration to facilitate cross-scale dependency modeling. Specifically, for the representation of the i-th time series $\mathbf{H}_i^R = \{\mathbf{h}_1^R, \mathbf{h}_2^R, \ldots, \mathbf{h}_m^R\} \in \mathbb{R}^{M\times D}$, which has been comprehensively extracted, the self-attention module employs linear projections to derive the queries, keys, and values denoted as $\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i \in \mathbb{R}^{M\times D}$. After this step, a scaling operation is utilized to compute the attention outputs, generating $\mathbf{O}_i$:

$$\mathbf{O}_i = \text{Attention}\left(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i\right) = \text{Softmax}\left(\frac{\mathbf{Q}_i \mathbf{K}_i^T}{\sqrt{D}}\right)\mathbf{V}_i.$$

The architecture of the attention layer encompasses Batch Normalization (BatchNorm), Dropout, and Residual Connection mechanisms. Subsequently, this architecture yields a representation, hereafter referred to as $\mathbf{H}_i^A = \{\mathbf{h}_1^A, \mathbf{h}_2^A, \ldots, \mathbf{h}_m^A\} \in \mathbb{R}^{M\times D}$.

Flatten Predicted Head: A linear layer is employed to amalgamate $\mathbf{H}_i^A$ to obtain the final prediction $\hat{\mathbf{Y}}_i = \{\hat{x}_{L+1}, \hat{x}_{L+2}, \ldots, \hat{x}_{L+S}\} \in \mathbb{R}^{1\times S}$

III. EXPERIMENTS

A. Experimental Settings

Datasets: We evaluate the performance of our proposed model on 7 benchmark datasets as follows: Weather, Traffic, Electricity, and four ETT datasets (ETTh1, ETTh2, ETTm1, ETTm2), which can be publicly available on Autoformer [8]. Among these datasets, Weather, Traffic, and Electricity are considered large datasets, with dataset sizes of 52,603, 17,451, and 26,211, respectively.

Comparison Methods: We choose state-of-the-art baseline models, including FITS [17], PatchTST [9], Crossformer [18], FEDformer [13], Autoformer [8],iTransformer [19], DLinear [12]. Each model adheres to a uniform experimental framework, characterized by a prediction interval, T, which is defined within the set $\{96, 192, 336, 720\}$. We aggregate the results of PatchTST, DLinear, and FEDformer from [9], and those of Crossformer from [19]. The results of FITS and iTransformer are sourced from their respective academic papers.

Model Variants: We set the look-back length L to 336 and assign scaling factors as follows: 3 for the Electricity, Traffic, ETTh1, and ETTh2 datasets; 2 for ETTm1 and ETTm2; and 4 for the Weather dataset. Typically, only seven patches are needed, except for Weather, where 15 patches are required. The number of layers in the residual dense and attention mechanisms is set to 1.

B. Main Result

The comprehensive results of the forecasting are delineated in Table I. The accuracy of the prediction outcomes is inversely proportional to the values of the mean squared error (MSE) and mean absolute error (MAE), with lower values signifying higher

TABLE II
COMPARISON OF MODELS' EFFICIENCY ON THE WEATHER, ELECTRICITY, AND TRAFFIC DATASETS. MACS DENOTES THE NUMBER OF MULTIPLY-ACCUMULATE OPERATIONS, PARAMS DENOTES THE NUMBER OF MODEL PARAMETERS, AND TIMES DENOTE THE INFERENCE TIME

	Weather			Electricity			Traffic		
Methods	MACS	Params	Times(ms)	MACS	Params	Times(ms)	MACS	Params	Times(ms)
CSformer	3.65M	104.08K	0.820	398.55M	493.01K	0.911	1.07G	493.01K	1.085
DLinear	4.25M	226.46K	0.200	72.09M	226.46K	0.207	195.42M	226.46K	0.205
PatchTST	389.77M	2.21M	2.063	5.95G	2.33M	5.077	16.23G	2.21M	12.971
iTransformer	111.12M	5.08M	2.377	1.63G	5.08M	2.407	5.74G	6.66M	2.939
FITS	711.36K	34.78K	1.414	10.59M	34.78K	1.753	29.11M	34.78K	2.199

TABLE III
ABLATION EXPERIMENTS. RES DENOTES THE RESIDUAL DENSE LAYER. ATT DENOTES THE ATTENTION LAYER

	Traffic(MSE)				Weather(MSE)			
Prediction length	96	196	336	720	96	196	336	720
w/o (Res+Att)	0.410	0.423	0.436	0.463	0.176	0.218	0.265	0.333
w/ Att	0.372	0.394	0.406	0.441	0.154	0.198	0.249	0.325
w/ (Res+Att)	**0.362**	**0.384**	**0.396**	**0.430**	**0.149**	**0.196**	**0.246**	**0323**

accuracy. Throughout all conducted experiments, our CSformer exhibits performance that is either on par with or surpasses that of the benchmarks.

Efficiency Analysis: We assess computational efficiency via three metrics: multiply-accumulate operations (MACs) [20], model parameters (Params), and inference time (Time), with methodology detailed at the site.[1] We analyze these metrics across Weather, Traffic, and Electricity datasets, with a lookback window of 336 time steps and forecast horizons of 96, 192, 336, 720 time steps. The results are shown in Table II. CSformer demonstrates a balance between computational efficiency and model complexity. Compared to DLinear, CSformer maintains enhanced performance while having a similar scale of parameters. Against models like PatchTST and iTransformer, CSformer achieves better Mean Squared Error (MSE) scores with fewer MACs, parameters, and a shorter inference time. CSformer achieves a speed that is 2-12x times faster than PatchTST. Although FITS operates with significantly fewer MACs and parameters, its reliance on Fourier transformation extends its reference time. In direct comparison, CSformer showcases an improved inference speed by 42%, 48%, and 51.9% on the large datasets of Weather, Electricity, and Traffic, respectively, indicating its superiority in processing efficiency and speed for complex data scenarios.

C. Ablation Study

Architecture Ablation: We conducted a series of ablation experiments to evaluate the effectiveness of our proposed Residual Dense Layer (Res) and Attention Layer (Att) within the CSformer framework. Table III summarizes the performance of our model under different configurations on both Traffic and Weather datasets across various prediction lengths. The results clearly show that removing both components (w/o Res+Att) results in the highest prediction errors. Introducing the Attention Layer alone (w/ Att) significantly reduces errors, indicating its positive impact on model performance. The best performance is achieved when both layers are incorporated (w/ Res+Att), demonstrating their synergistic effect. For instance, in the Traffic dataset, the model's error decreases from 0.410 to 0.362 for a prediction length of 96, and similar improvements are observed for other lengths. The Weather dataset exhibits a consistent trend, with the error decreasing from 0.176 to 0.149 for the same prediction length when both layers are used.

[1]https://deci.ai/blog/measure-inference-time-deep-neural-networks/

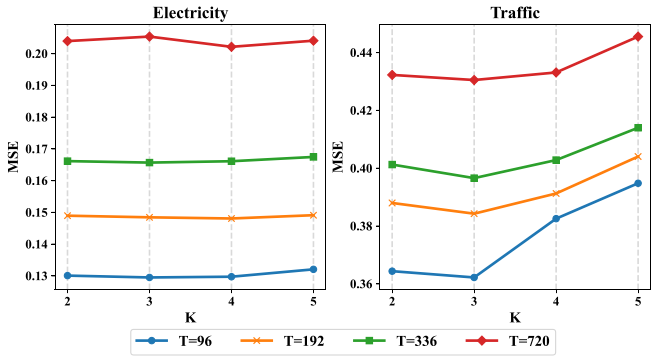

Fig. 2. Sensitivity analysis of the hyper-parameter $scale$.

Varying $\boldsymbol{scale}$ of Multi-scale Patching: The level of multi-scale patching $scale$ determines both the number of patches and the degree of refinement of encoding time series. To study the impact of the level of multi-scale patching, we conduct the experiments on two datasets (Electricity and Traffic) with $scale \in \{2, 3, 4, 5\}$. Fig. 2 shows that while CSformer achieves similar MSE scores on the Electricity and Traffic datasets when varying the level of multi-scale patching, it achieves the best MSE scores with $scale = 3$ specifically for the Traffic dataset.

IV. CONCLUSION

Our study presents a novel approach for improving time series forecasting with Transformer-based models, addressing the challenges posed by the sparse semantic relationships and lower information density in time series data. By introducing a multi-scale patching technique that utilizes cross-scale attention interaction, our method not only significantly reduces the number of patches required but also enhances computational efficiency, achieving speeds 2 to 12 times faster than the best Transformer-based models. Our results, which either surpass or closely approach those of current methods on benchmarks, demonstrate the effectiveness and efficiency of our approach, suggesting a promising direction for future research in optimizing deep learning models for time series analysis.

REFERENCES

[1] Q. Gao, F. Zhou, K. Zhang, F. Zhang, and G. Trajcevski, "Adversarial human trajectory learning for trip recommendation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 4, pp. 1764–1776, Apr. 2023.

[2] Q. Gao, X. Wang, C. Liu, G. Trajcevski, L. Huang, and F. Zhou, "Open anomalous trajectory recognition via probabilistic metric learning," in *Proc. 32nd Int. Joint Conf. Artif. Intell.*, Aug. 19–25, 2023, pp. 2095–2103.

[3] E. Koç and A. Koç, "Fractional fourier transform in time series prediction," *IEEE Signal Process. Lett.*, vol. 29, pp. 2542–2546, 2022.

[4] B. Zhao, H. Xing, X. Wang, Z. Xiao, and L. Xu, "Classification-oriented distributed semantic communication for multivariate time series," *IEEE Signal Process. Lett.*, vol. 30, pp. 369–373, 2023.

[5] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Adv. Conf. Neural Inf. Process. Syst.*, 2017 .

[6] J. Li, X. Hui, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. 35th AAAI Conf. Artif. Intell.*, 2021, pp. 11106–11115.

[7] S. Liu et al., "Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting," in *Proc. Int. Conf. Learn. Representations*, 2021.

[8] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. Int. Adv. Conf. Neural Inf. Process. Syst.*, 2021.

[9] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with transformers," in *Proc. Int. Conf. Learn. Representations*, 2023.

[10] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2021.

[11] T. Brown et al., "Language models are few-shot learners," in *Proc. Int. Adv. Conf. Neural Inf. Process. Syst.*, 2020.

[12] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are transformers effective for time series forecasting?," in *Proc. AAAI Conf. Artif. Intell.*, 2023.

[13] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn.*, 2022.

[14] L. Han, H.-J. Ye, and D.-C. Zhan, "The capacity and robustness trade-off: Revisiting the channel independent strategy for multivariate time series forecasting," *IEEE Trans. Knowl. Data Eng.*, early access, May 13, 2024, doi: 10.1109/TKDE.2024.3400008.

[15] A. van den Oord et al., "Wavenet: A generative model for raw audio," in *The 9th ISCA Speech Synth. Workshop*, 2016, Art. no. 125.

[16] A. Borovykh, S. Bohte, and C. W. Oosterlee, "Conditional time series forecasting with convolutional neural networks," 2017, *arXiv:1703.04691*.

[17] X. Q. Xu Zhijian and Z. Ailing, "FITS: Modeling time series with 10 k parameters," in *Proc. Int. Conf. Learn. Representations*, 2024.

[18] Y. Zhang and J. Yan, "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2023.

[19] Y. Liu et al., "itransformer: Inverted transformers are effective for time series forecasting," in *Proc. Int. Conf. Learn. Representations*, 2024.

[20] X. Cao et al., "Massive access of static and mobile users via reconfigurable intelligent surfaces: Protocol design and performance analysis," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 4, pp. 1253–1269, Apr. 2022.